École Polytechnique Fédérale de Lausanne
Semester project
Master of Civil Engineering, Spring 2024-2025

A real-time heuristic for the railway timetable rescheduling problem

Julien Ars

Under the supervision of Léa Ricard, Negar Rezvany and Prof. Michel Bierlaire (Transp-OR)

A suburban train in the Vaud canton.

Contents

1 Introduction

One of the main challenges in railway operations is maintaining an efficient and reliable service. When disruptions occur, such as track closures or speed restrictions due to maintenance or external events, operators usually need to choose between delaying, cancelling, or rerouting trains to limit the consequences on the rest of the network and ensure that the maximum number of passengers reach their destinations. This decision-making process, previously entirely delegated to human operators in control centres, could be automated using the fields of operations research and optimisation. In this context, multiple approaches have been proposed to solve the problem of train rescheduling efficiently. Exact approaches can solve small instances, but for real-life large-scale networks, they often prove resource-intensive and require long computation times. Heuristic solutions, on the other hand, can offer relatively good solutions within reduced computation times. Since disruptions can happen unexpectedly, the latter option will be the focus of our project.

Heuristics research is a vast topic, but in the case of routing or scheduling problems, Adaptive Large Neighbourhood Search (ALNS) has distinguished itself as a powerful metaheuristic framework. ALNS combines multiple heuristic approaches and iteratively applies them to improve solution quality. Its adaptability allows it to explore a large solution space efficiently, making it particularly suitable for complex and dynamic problems. Furthermore, ALNS is known for its flexibility in integrating problem-specific knowledge, which can significantly enhance its performance in real-time applications.

After building a mixed-integer linear programming (MILP) model (Ricard and Bierlaire 2025), the final aim of the project is to develop a real-time ALNS heuristic capable of solving problem instances in the RER Vaud. The RER Vaud is the suburban network serving the canton of Vaud in Switzerland. Centred around the city of Lausanne, it has four main branches reaching all corners of the canton (see Figure 1). Some of its routes last for more than two hours (line R4 from Le Brassus to Bex, 139 min), while others operate in around a quarter of an hour (line R7 from Palézieux to Vevey, 17 min) (Wikipedia 2025). The trains operate on the national railway network, including the busy Geneva–Lausanne line, and as such, a suboptimal solution to a disruption could have large impacts.

This document presents our personal contribution to this project. After a brief review of relevant literature, we introduce the mathematical definition of the rescheduling problem and the modelling of the project, before elaborating on our methodological propositions. We also discuss the results obtained from our implementation and evaluate its performance in terms of solution quality and computational efficiency. For computational and confidentiality reasons, we test our proposals on a smaller instance inspired by the Dutch network.

Figure 1: RER Vaud Network (Wikimedia Commons 2024)

2 Relevant literature

In this section, we discuss literature associated with the subject across three different axes: first, previous research at Transp-or; then, an introduction to the ALNS framework; and thirdly, a review of recent work in the literature about ALNS. Finally, we present the contribution of the present work.

2.1 Inspiration for the project

The project builds upon previous research at the Transp-OR laboratory, in particular works by Stefan Binder, Maknoon, and Bierlaire:

In *"The multi-objective railway timetable rescheduling problem" (2017c)*, they introduce a time-indexed formulation for the same problem, implement a MILP mathematical model, and then test it on a small instance inspired by the Dutch network.

In *"Exogenous priority rules for the capacitated passenger assignment problem" (2017b)*, they propose, for the passenger assignment problem, an exogenous framework that relies on priority lists defined before any assignment, ensuring the outcome of capacity constraint resolutions remains independent of the algorithm used or its implementation. It is an interesting framework to make the solution closer to the User Equilibrium paradigm, where each passenger is assigned their preferred route and not the one that minimises the total cost for the system.

In *"Efficient exploration of the multiple objectives of the railway timetable rescheduling problem" (2017a)*, they introduce a heuristic based on the Adaptive Large Neighbourhood Search (ALNS) paradigm. The neighbourhood operators defined correspond to real-life decisions (e.g. cancelling/delaying trains completely or after a given station, rerouting them between neighbouring stations, adding an emergency train or bus). They allow the solutions to be infeasible, but in that case the next operator is forced to resolve the infeasibility. Dealing with multiple objectives, they keep an archive of Pareto-dominant solutions, and periodically choose a random solution from this archive.

Later, in *"Passenger-centric timetable rescheduling: A user equilibrium approach" (2021)*, they refine their previous work and propose another algorithm based on ALNS. While they keep the same operators, they can apply different strategies when defining their parameters: local search (find the locally best solution for one objective), random selection, disruption mitigation (adding an emergency bus specifically between the two ends of the disruption), or feasibility restoration. The operators are grouped into sets (for example, *Cancelling operators*) which can be chosen with equal probability, and then selection within the set is based on past performance. Passenger assignment is done as suggested in (2017b).

2.2 The Adaptive Large Neighbourhood Search (ALNS) metaheuristic

Adaptive Large Neighbourhood Search (ALNS) is a paradigm introduced in Ropke and Pisinger (2006), which extends the Large Neighbourhood Search metaheuristic. In Pisinger and Ropke (2019), the same authors describe the metaheuristic (and its parent, the Large Neighbourhood Search - LNS) again, with a discussion of recent developments. We take from this document the following concepts and definitions:

A Neighbourhood Search (NS) algorithm is a best improvement algorithm that will, starting with an initial solution, try to find a better solution in the neighbourhood, and iterate until it finds a local optimum.

A Large Neighbourhood Search (LNS), introduced in Shaw (1998), instead has a destroy and repair mechanism. It will try to delete a part of the solution and then repair it (using another heuristic, for example). This will be followed by an acceptance mechanism, which can decide whether to keep or not the resulting solution, meaning that a tentative solution could be kept even if it does not have a better optimal value than the current accepted solution. One common such mechanism is simulated annealing, where a non-improving solution x' can be accepted with a probability $e^{-\left(c(x')-c(x)\right)/T}$, where x is the current best solution, $c(x)$ is its cost (objective value in a minimisation problem) and $T > 0$ is a temperature that will decrease over the number of iterations. Here, the neighbourhood of a solution is the set of solutions that can be reached by this *destroy & repair* mechanism, and we now only explore a sample of it. Special attention should be given to the *degree of destruction* at each destroy step.

In ALNS, multiple *destroy* and *repair* operators are considered in the same algorithm, and the probability of using a given operator varies dynamically over the iterations. It typically depends on a score reflecting the

past performance of that operator, making the algorithm adaptive to the current problem instance. In addition, the score can be applied to individual *destroy* or *repair* operators, their combinations, or a single operator that takes care of the complete *destroy & repair* mechanism. In addition to the solution quality, the score sometimes also includes the time consumption, ensuring the algorithm does not overly favour complex operators that produce better solutions but at the cost of lengthy iterations. The extension of LNS that includes multiple *destroy* and *repair* operators selected randomly without adaptation of the weights is sometimes called large multiple-neighbourhood search (LMNS).

For the operators, the adaptive element of the metaheuristic allows the inclusion of a diversity of them, as the algorithm will in itself choose the best ones for the current instance. In terms of *destroy* operators, we can distinguish *random*, *critical*, *related* or *history-based* concepts. In terms of *degree of destruction*, multiple destroy operators with different degrees of destruction could be included and the choice left to the adaptive mechanism. Well-performing heuristics for the considered problem constitute most of the *repair* operators. Other possibilities include approximation or exact algorithms, for example using a MIP solver.

For the acceptance mechanism, in addition to simulated annealing, sometimes also called Metropolis, other possibilities include a greedy mechanism (which accepts a solution only if it improves the objective function), record-to-record (accept x' if $(c(x') - c(x))/c(x') < R$, with R a predetermined threshold) and its variant, the threshold acceptance (where R decreases at each iteration). For multi-objective problems, the concept of Pareto dominance accepts a new solution if it improves at least one objective without worsening the other objectives (Windras Mara et al. 2022).

2.3 State-of-the-art

The focus of this project being on the solution approach to the problem, we now conduct a literature review focused on work done in the field of ALNS implementation for various problems, including those outside railway problems.

Windras Mara et al. (2022) made a comprehensive review of the current developments of ALNS. They found that nearly all studies (99%) used a roulette wheel mechanism for selecting the next operator; that the simulated annealing criterion was the most used (81%) acceptance criterion, followed by the greedy mechanism (8%, also known as hill-climbing) and record-to-record (8%); and that for the termination criterion, the number of iterations was the most popular (75%), followed by a running time limit and the number of non-improving iterations (19%), and finally the annealing temperature (10%). They also listed common destroy and repair operators, noting that it is frequent to define operators specific to the problem at hand. Regarding hybridisation strategies between ALNS and other methods, they found that adding a post-optimisation strategy, for example based on exact methods or a local search procedure, often improved the solution quality.

Turkeš, Sörensen, and Hvattum (2021), through a review of multiple studies, researched the efficiency of the *adaptive* layer of ALNS. One of their major outcomes is that this adaptive layer was most useful when there was sufficient diversity in the instances for which the heuristic is being developed, but also in the operators considered, meaning operators have the ability to work better in some instances than in others. Proper consideration of the computation time of the operators also helped to increase the positive effect of the adaptive character of the metaheuristic, as well as compensating for the initial advantage one heuristic could have from being selected first (since improving the solution is easier in the first iterations). The paper also pointed out that the adaptive process did not compensate efficiently for the presence of non-efficient heuristics in the destroy and repair operator sets, and that a thorough selection of the set of operators increased performance more than simply adding an adaptive process.

Barrena et al. (2014), included in the previous analysis, applied an ALNS to a train scheduling problem on a single-line rapid rail transit system. Responding to a dynamic demand curve, the objective was to minimise mean passenger waiting time at the departure station. Their destroy operators removed trains, and repair operators reinserted them, either randomly, based on the interval with the next train, or based on the demand curve. In this example, the effect of the adaptive layer is very small, with an average improvement in solution quality of 0.04% when compared with fixed scores for the operator (Turkeš et al. 2021).

In Thomas and Schaus (2018), the authors introduced a generic ALNS framework for use in multiple problems, designing some operators that assume a given problem configuration and letting the adaptiveness of the algorithm take care of efficient operators for the current instance. They also proposed a different weight adjustment mechanism, which considered the local (in terms of time) and total efficiency of the operator. In this

case, the impact of adaptiveness is better, with the maximum in the previous efficiency analysis at 15.46 % improvement (Turkeš et al. 2021). They propose 30 destroy operators and 36 repair operators, although some differ only by the value of their parameters.

Mairy, Deville, and Van Hentenryck (2011) introduced an LNS algorithm, where the destroy operator is based on reinforcement learning agents. One relevant point of the proposition is that the impact of the reward was increased at each iteration, giving more importance to the results later in the search. This echoes remarks in Turkeš et al. (2021), where multiple studies are cited for which the final probabilities of the operators were different in multiple runs of the same instance, which could be related to the weight a heuristic can gain by being selected first.

Santini, Ropke, and Hvattum (2018) made a comparison of acceptance criteria. Regarding simulated annealing, they propose a formula to determine the temperature. They also introduce an automatic parameter tuning algorithm. For the comparison, they test 10 different acceptance criteria on ALNS algorithms for 3 common problems: capacitated minimum spanning tree problem, quadratic assignment problem, and capacitated vehicle problem. For the latter, they also test the 10 acceptance criteria on one LNS algorithm. For each criterion, multiple variants are tested, for example a linear decreasing temperature and an exponentially decreasing temperature. As a result, they recommended: (1) To try the 3 acceptance criteria that are Simulated Annealing, Threshold Acceptance and Record-to-Record Travel, with a preference for Record-to-Record Travel. In this case, the latter is defined as Threshold Acceptance but comparing the tentative solution to the best solution instead of the current solution, a different definition than we previously stated. (2) To use a linear function for the parameter (for example, the temperature). However, their results were different for each considered problem.

Liao, Zhang, and Wei (2020) implemented a two-stage solution strategy for the multi-objective problem in a meal delivery routing context. The 3 objectives considered were delivery duration, carbon footprint and equalisation of the orders' distribution between the riders. The strategy implemented was composed of two stages, the first one being a genetic algorithm to determine the optimal number of riders, followed by ALNS to solve the delivery routing of each rider. They used 3 destroy operators: priority removal (based on assigned priority of the orders), least profit removal and shaw removal (remove similar customers); and 2 repair operators: cheapest insertion (insert the orders where it is the cheapest, starting with the most expensive one) and regret insertion (insert the orders where it is the cheapest, starting with the order with the biggest difference between its best and second-best positions). The multiple objectives were combined into a single objective function for the ALNS.

Rifai, Nguyen, and Dawal (2016) implemented a multi-objective ALNS for the distributed re-entrant flow shop scheduling problem, a kind of job shop scheduling problem. In contrast with the previous paper, the multiple objectives were considered using a Pareto set of non-dominated solutions. During the execution of the ALNS, an archive of non-dominated solutions was kept, and when the current solution was not accepted by the Metropolis criterion, a solution was selected randomly from this archive. The destroy operators included random destroy and other operators specific to the problem, some of them targeting improvement for one of the objectives. The level of destruction is controlled by a linearly decreasing parameter, that goes from 50% to 10% of the solution. In terms of repair operators, they used archive and elitist based operators (where deleted parts of the solution are filled with parts from respectively other non-dominated solutions and the solution with the minimal sum of the objective functions), local search and random repair. Another specificity was the consideration of diversification in the score of the operators, using Euclidean distance in the 3D space of the 3 objective functions.

We summarise the operators for the ALNS implementations discussed above in Table 1.

Article	Destroy operators	Repair operators
Barrena et al. 2014	Random Min. interval Min. demand	Random Max. interval Near max. demand
Thomas et al. 2018	Random Sequential (Reverse) Propagation Guided Value Guided K opt Precedency based Cost impact	By variable: FirstFail Conflict Ordering Weighted Degree By value: Min, Max, Median, Random, Sticking
Liao et al. 2020	Priority Least profit Similarity	Cheapest Regret
Rifai et al. 2016	Random Factory usage Objective-based	Archive based Elitist based Local search Random

Table 1: Operators used

2.4 Our contribution

We aim to propose a heuristic based on the ALNS framework with innovative operators. Expanding on the work in S. Binder et al. (2021), we present an efficient algorithm to solve passenger assignment dynamically at each iteration. We then define operators for the train rescheduling problem by drawing inspiration from the operators seen in the literature. Finally, we review the performance of our operators using an external framework and propose next steps to improve the performance of the heuristic.

3 Mathematical modelling

The problem statement, solution representation and objectives were defined in Ricard et al. (2025). For completeness, we provide a summary of the most relevant parts.

3.1 Problem statement

The railway timetable rescheduling problem is defined as follows:

Input

- Given a railway network with a set of stations S, each with a set of node tracks P_s $\forall s \in S$, and a set of section tracks Q between stations and/or bifurcations.
- Given a planned schedule, consisting of a set of predefined trains K, each composed of a sequence of stations and bifurcations with given arrival and departure times.
- Given passenger groups $g \in G$, each with a desired origin station o_g, destination station d_g, desired departure time t_g and size n_g.
- Given a disruption with known duration, impacting one or multiple section tracks, either partially (reduced speed) or completely.

Output Obtain a new schedule for the trains that minimises passenger cost, operational cost, and deviation from the original schedule.

3.2 Graphical representation

The problem is represented in an Event-Activity graph $G(E, A)$ in which each train is represented by its path. The arcs $a \in A$ of this graph represent an activity the train could perform (*running* on a section track, *pass-through* or *waiting* at a node track, or *starting* or *ending* at a station), while nodes $e \in E$ represent an event (for example: *train 15 arriving at station A on Node Track A_1*). Table 2 presents the events and Table 3 the activities present in the graph, with a short description of them and the characteristics associated with each. In Table 3, we also indicate which time constraints exist on the y variable for these activities, with y_i the time at which the activity starts, y_j the time at which the activity ends, *min_tt(q, k)* the minimum travel time for train k on section track q, and *min_wt* the minimum waiting time. The node track and station information is associated with the event, while the section track is associated with the corresponding *running* activity. As a result, for a given section track, the *running* activities need to be duplicated for every possible node track at its origin and end station.

Event type	Description	Characteristics
Train origin	Fictive node for the origin of the train	Train $k \in K$
Train destination	Fictive node representing the destination of the train	Train $k \in K$
Departure	Train leaves a station	Station $s \in S$ Node track $p \in P_s$ Train $k \in K$
Arrival	Train arrives at a station	Station $s \in S$ Node track $p \in P_s$ Train $k \in K$

Table 2: Events of the Event-Activity graph

Activity type	Description	Characteristics	Time Restrictions
Running	Train running between two stations on a given section track	Section track $q \in Q$ Train $k \in K$, Events i, j	$y_j - y_i > min\_tt(q,\ k)$
Pass-through	Train passing through a station without stopping	Train $k \in K$, Events i, j	$y_j = y_i$
Waiting	Train stopping at a station	Train $k \in K$, Events i, j	$y_j - y_i \geq min\_wt$
Starting	Train starts at a station	Train $k \in K$, Events i, j	/
Ending	Train ends (leaves the system)	Train $k \in K$, Events i, j	/

Table 3: Activities of the Event-Activity graph

Each solution is represented by the following decision variables: $\boldsymbol{x}$ (binary), which represents whether a given arc is used; $\boldsymbol{y}$ (continuous), which represents at what time each event happens; and $\boldsymbol{z}$ (binary), which represents whether each event is cancelled.

3.3 Constraints

A valid solution then needs to satisfy the following constraints, in addition to the time precedence constraints presented in Table 3:

1. Only one starting activity per train.
2. Flow conservation constraint at each event: any incoming activity is matched with an outgoing activity, or the event is cancelled.
3. Trains can only be cancelled at stations with a shunting yard.
4. When train k is on a section track:
   - No train runs in the other direction.
   - No train enters or leaves the track in the same direction in the $h(k)$ minutes before train k, with $h(k)$ the headway of train k.
5. When train k is on a node track:
   - No train enters the node track.
   - No train enters the node track until *min_sep* minutes after it leaves.
6. Events do not occur before their scheduled time, nor are they delayed by more than 30 minutes.

In addition, we only consider the following recovery decisions: cancelling, delaying, track change, stop removal or stop addition. This list could, however, later be extended to include rerouting, short-turning, and emergency trains.

3.4 Objectives

We define the following three objective functions:

1. The inconvenience for passengers, which is the sum of the inconvenience for each passenger: $f_p = \sum_{g \in G} f_{p,g} \times n_g$. This is in turn defined as the opposite of their utility, and we estimate it using the following formula:

$$f_{p,g} = \mathrm{VT}_g + \beta_1 \times \mathrm{WT}_g + \beta_2 \times \mathrm{NT}_g + \beta_3 \times \mathrm{ED}_g + \beta_4 \times \mathrm{LD}_g, \tag{1}$$

   where, for passenger group g:
   - VT_g is the in-vehicle time.

- WT_g is the waiting time, defined as the time the passenger group spends in a train that waits at a station.
- NT_g is the number of transfers.
- $\mathrm{ED}_g = \max(0, t_g - t)$ is the 'early departure' time difference between the desired start time and the actual start time.
- $\mathrm{LD}_g = \max(0, t - t_g)$ is the 'late departure' time difference.

For the weights, we take the following values: $\beta_1 = 2.5$ min/min, $\beta_2 = 10$ min/transfer, $\beta_3 = 0.5$ min/min, and $\beta_4 = 1$ min/min (Ricard et al. 2025). The determination of the variables VT_g, WT_g, NT_g, ED_g and LD_g requires solving a passenger assignment subproblem (see Section 4.1).

2. The cost for the railway operator, which is assumed to be proportional to the distance travelled by the trains, and is defined as:

$$f_o = c_{\mathrm{km}} \times \sum_{q \in Q} \left(d_q \times \sum_{a \in A^q} x_a \right) \tag{2}$$

where:

- c_{km} is the cost per km travelled by a train.
- d_q is the distance of section track q.
- A^q is the set of (running) activities using section track q.
- x_a is the decision variable associated with activity a happening.

3. The deviation from the regular schedule. For the recovery decisions considered, it is measured by:

$$f_d = \delta_1 \times \sum_{e \in E} \Delta t_e \; z_e + \delta_3 \times \sum_{e \in E} (y_e - \overline{y}_e) + \delta_5 \times N_{\text{skipped}} + \delta_6 \times N_{\text{added}} + \delta_7 \times N_{\text{Changed track}} \tag{3}$$

where:

- δ_1, δ_3, δ_5, δ_6, δ_7 are weights.
- E is the set of all events.
- Δt_e is the time difference between the scheduled time of the last event of train k and the scheduled time of event e, where k is the train associated with event e.
- z_e is the decision variable that determines whether event e is cancelled.
- y_e is the time at which event e happens in the solution.
- $\overline{y}_e$ is the time at which event e was scheduled to happen.
- N_{skipped} is the number of stops skipped.
- N_{added} is the number of stops added.
- $N_{\text{Changed track}}$ is the number of times a train changed section track.

In the MILP, the three objectives are considered using an ϵ-constrained method, where the solution is optimised for each objective in order, adding a constraint on the next objectives. The first objective is optimised without any constraint, the second one is optimised with a constraint on the first one (such that the difference in optimal value is smaller than some ϵ), and the third one is optimised with similar constraints on the first two.

For the heuristic, we try to determine the Pareto frontier, where it is not possible to improve the solution for one objective without penalising at least one other objective. As such, we keep multiple solutions for which we have not found better (Pareto dominant) solutions.

4 Methodology

4.1 Passenger assignment subproblem

One of the main components of any heuristic algorithm is the evaluation of the objective function(s), which are required to assess the quality of the solution. In our multi-objective problem, the first objective, passenger satisfaction, is the most complex to compute. Indeed, in this formulation, the passenger cost depends on the passenger's path within the system, meaning we need to solve some sort of passenger assignment. The next two objectives (operational costs and deviation from the original schedule) are derived directly from the variables x, y and z that represent our solution, and as such can be computed in a straightforward way.

In the passenger assignment subproblem, we try to assign to each passenger group a path across our system, or in other words, determine which trains a passenger is taking. The passenger cost is then computed based on the different activities a passenger undertakes along their path, including *running* (being in a train between stations), *dwelling* (being in a train waiting at a station), *transferring* (leaving their current train and waiting for another one), *access* (boarding their first train), *egress* (leaving the network after arriving at their destination), and *penalty* (when the system has no solution for this passenger group in the time horizon, which is modelled with a high, fixed penalty cost, possibly corresponding to offering a bus or a taxi) activities.

In order to determine the activities each passenger group will undertake, we build a passenger graph on top of the train graph. Using the x variable to know which trains are running and the y variable to know when they are arriving at and leaving a station (and thus determine on-boarding, off-boarding or transfer opportunities), we build a reduced event-activity graph, different from the 'train' graph, that represents the paths available to the passengers, from their origin to their destination. For each valid *Running* activity of the train graph, we generate a *Running* activity in the passenger graph, while for each *waiting* or *pass-through* train activity we generate a corresponding *Dwelling* activity. Each time a train stops at a station, corresponding *Transferring*, *Access* and *Egress* activities are generated for any valid combinations. The *Penalty* arcs are enumerated for each group, from its origin event to its destination event. In order to make the graph as compact as possible, we group the egress arcs and the destination events per station. Table 4 presents the different activity types that compose this graph. The number of activities of each type is expressed in terms of the number of corresponding activities in the train graph for which $x = 1$ ($N_{\text{train running}}$, N_{waiting} and $N_{\text{pass-through}}$), the number of passenger groups (N_{groups}) and the number of stations that serve as destination to at least one passenger group ($N_{\text{end stations}}$). *max_trans* and *min_trans* are problem variables defining the maximum and minimum duration of a transfer, y_i^k defines the time at which event i of train k happens, and t_g is the desired start time of passenger group g.

Activity type	Events type	Number	Time Restrictions	Cost
Running	Train Dep. → Train Arr.	$N_{\text{train running}}$	$y_j^k - y_i^k > 0$	$y_j^k - y_i^k$
Dwelling	Train Arr. → Train Dep.	$N_{\text{waiting}} + N_{\text{pass-through}}$	$y_j^k - y_i^k > 0$	$\beta_1 \cdot \left(y_j^k - y_i^k\right)$
Transferring	Train Arr. → Train Dep.	$\sum_{s \in S} N_{\text{waiting}}^2 - N_{\text{waiting}}$	$y_j^{k'} - y_i^k < max\_trans$ $y_j^{k'} - y_i^k > min\_trans$	$\beta_2 + \left(y_j^{k'} - y_i^k\right)$
Access	Pass. Org. → Train Dep.	N_{groups}	/	$\beta_3 \cdot \max(0, t_g - y_i^k)$ $+\beta_4 \cdot \max(0, y_i^k - t_g)$
Egress	Train Arr. → Pass. Dest.	$N_{\text{end stations}}$	/	0
Penalty	Train Arr. → Pass. Dest.	N_{groups}	/	T

Table 4: Activities of the passenger graph

If we do not consider capacity constraints, the assignment is trivial, resulting in selecting for each passenger group the path that would minimise its cost (*its shortest path*), without interaction with the other passengers. However, one goal of the passenger assignment should be to closely mimic the real behaviour of passengers. Even in open systems[1], there may be situations where people cannot board a train because it is full. As such, we try to implement a capacity constraint on the train, resulting in the potential need to adjudicate the place on a given train between different groups. In the MILP formulation, the programme would make the choices that minimise

[1] An open system is a train system where passengers can freely choose their itinerary, for example in most Swiss trains. A closed system is a train system where passengers need to book their place on a specific train, for example in French TGVs.

the total ('systemic') cost (resulting in implementing the *System Optimum* paradigm (S. Binder et al. 2021)). In an open system, this is unrealistic, as it could assume passengers are leaving the train of their own accord in order to let another group board. As such, our solution will rather assume users regulate themselves (the *User Equilibrium* paradigm (S. Binder et al. 2021)), and try to represent such behaviour. Different approaches are possible to solve this. As we already discussed in Section 2.1, Stefan Binder et al. (2017b) introduced a framework that relies on predefined priority assigned to the passengers to adjudicate the presence in the train. We introduce another approach which will always adjudicate in favour of the passengers already in the train, which we believe makes it more realistic.

We therefore introduce two algorithms that can solve the passenger assignment, based on the two approaches previously mentioned. We denote them respectively *Exogenous Priority* (Algorithm 1) and *Presence in the train* (Algorithm 2). Both algorithms require a `ShortestPath` solution. We propose using Dijkstra's labelling algorithm. Since we have grouped the end nodes by stations, we can reduce the number of times the labelling process needs to be done by running it backwards, from the end nodes. As such, we solve the shortest path at once for all groups with the same destination.

Algorithm 1: Passenger Assignment (Exogenous Priority)

Input: A : arcs list; G: passenger groups list;
p: passenger groups' priorities;
n_g: number of passengers in group g;
q_a: capacity of arc a
Output: c_g: path cost of group g;
x_a^g: binary variable, true if group g uses arc a

1 Order G according to p;
2 $\boldsymbol{c}, \boldsymbol{x} \leftarrow$ `ShortestPath(`G, A`)`;
3 $u_a \leftarrow 0 \quad \forall a \in A$;
4 **while** $G \neq \emptyset$ **do**
5 $g \leftarrow$ `first(`G`)`;
6 **for** $a \in A$ **do**
7 $\hat{u}_a \leftarrow u_a + n_g * x_a^g$;
8 **if** `any(`$\hat{u}_a > q_a$`)` **then**
9 $A \leftarrow A \setminus \{a \in A : \hat{u}_a > q_a\}$;
10 $G' \leftarrow \{g \in G : x_a^g = 1\}$;
11 $\boldsymbol{c}, \boldsymbol{x} \leftarrow$ `ShortestPath(`G', A`)`;
12 **else**
13 $\boldsymbol{u} \leftarrow \hat{\boldsymbol{u}}$;
14 $G \leftarrow G \setminus \{g\}$;

Algorithm 2: Passenger Assignment (Presence in the train)

Input: A : arcs list; G: passenger groups list;
p: passenger groups' priorities;
n_g: number of passengers in group g;
q_a: capacity of arc a;
t_a: time at which activity a starts
Output: c_g: path cost of group g;
x_a^g: binary variable, true if group g uses arc a

1 Order A according to t;
2 $A^g \leftarrow A \quad \forall g \in G$;
3 $\boldsymbol{c}, \boldsymbol{x} \leftarrow$ `ShortestPath(`G, A`)`;
4 $u_a \leftarrow \sum_{g \in G} n_g * x_a^g \quad \forall a \in A$;
5 $A_{\text{over}} \leftarrow \{a \in A : u_a > q_a\}$;
6 **while** $A_{over} \neq \emptyset$ **do**
7 $a \leftarrow$ `first(`A_{over}`)`;
8 $G' \leftarrow$ `GroupsToRemove(`$a, \boldsymbol{x}, p$`)`;
9 **for** $g \in G'$ **do**
10 $A^g \leftarrow A^g \setminus \{a\}$;
11 $c_g, \boldsymbol{x^g} \leftarrow$ `ShortestPath(`g, A^g`)`;
12 $u_a \leftarrow \sum_{g \in G} n_g * x_a^g \quad \forall a \in A$;
13 $A_{\text{over}} \leftarrow \{a \in A : u_a > q_a\}$;

Algorithm 1 is similar to the one presented in Stefan Binder et al. (2017b); we have adapted it only to benefit from the computational trick presented above. As such, after computing the shortest path for every group at the start (the computational cost of which is minimal, since the calculation needs to be done only once for any end station), we load the groups in order of their priority onto the arcs, updating the arcs usage vector u_a(**line 7**). If at least one arc is overused (**line 8**), it is removed from the list of arcs and the shortest path is recomputed for all the groups not yet loaded which were using this arc (**lines 9-11**). The algorithm ends when all groups are loaded onto the arc.

In Algorithm 2, instead of an exogenous priority, the decision of which group to remove from an overused arc has to be made at every iteration. After ordering the arcs according to their start time (**line 1**) and creating a list of which arcs each group can use (**line 2**), we start by computing the shortest path for each group (**line 3**) and loading the arcs accordingly (**line 4**). Then, we compute which arcs are overused (**line 5**), and take the first such arc (according to its start time t_a; **line 7**). The function `GroupsToRemove` (**line 8**) is one that, given an arc a, the paths of the groups $\boldsymbol{x}$ and the groups' priority p, will load the train in the order the passenger groups

board it and return the groups that would make it full at the given arc a. If there are multiple groups trying to board at the same station, it uses the assigned priority p to make the determination between them. **Lines 9 to 12** will then, for each of these groups, remove the arc from the arcs they can use and recompute their shortest path. This process can be slightly sped up by grouping together groups in G' if they currently have the same list of arcs A^g and computing their shortest path together. We end with updating arc usage (**line 13**) and the list of overused arcs (**line 14**). We restart at **line 7** while there are still some arcs overused (**line 6**).

4.2 Heuristic operators

The next step in building our heuristic is to define the different operators we will use. We require *destroy* operators, which remove part of the solution, and *repair* operators, which reconstruct this part of the solution. In our case, the *destroy* operators select trains for which the *repair* operators will change the solution.

For the rest of the ALNS mechanism, we rely on the pre-existing framework of `biogeme-optimisation`[2]. This framework keeps an archive of Pareto non-dominated solutions and of considered solutions, as well as a score for each operator and a size n parameter for any solution in the Pareto non-dominated set. It will make N attempts to improve the Pareto set by finding new non-dominated solutions. For each attempt, a starting solution is selected from the archive of Pareto non-dominated solutions, and operators are applied to this solution. If the resulting solution was already present in the considered set of solutions, if it is invalid, or if it is dominated by at least one of the solutions in the Pareto set, the score of the operator is decreased by 1, the size n of the starting solution is increased by 1, and a new operator is applied to the starting solution (up to 5 operators per attempt). On the other hand, if the returned solution is non-dominated, the solution is added to the Pareto set, the size parameter n of the starting solution is reset to 1, and the score of the operator is increased by 1.

We have defined 13 destroy operators and 6 repair operators. Nearly all 78 possible pairs can be selected (the only exception is the **Cancelled totally** and **Cancel completely** pair), and the score is associated with the pair of operators (77 different scores are tracked).

4.2.1 Destroy Operators

Our destroy operators select trains for which we want to change the selection. We need them to be diverse to efficiently explore the solution space. As such, we use the following strategies:

Random We select n random trains.

Most used We select the n trains that are most used by passengers, according to the passenger assignment algorithm (see Section 4.1). This can be done based on the mean or the maximum usage of the train along its route.

Less used We select the n trains that are least used by passengers. Similarly, we define 'mean' and 'min' versions of this operator.

Station We select n stations, and return all trains that pass through them in random order.

Section We select n random sections *(one section being a pair of stations for which there exists at least one track between them)*, and return all trains on them in random order.

Section Track We select n random section tracks, and return all trains on them in random order.

Disruption We select n trains randomly from those that run through the disrupted tracks during the disruption.

Disrupted tracks We select n trains randomly from those that run through the disrupted tracks, even if they pass before or after the disruption.

Disrupted sections We select n trains randomly from those that run through a section with at least one disrupted track, even if they pass before or after the disruption.

Cancelled We select n trains from those that are cancelled for at least a portion of their trip.

[2]Michel Bierlaire, biogeme-optimisation, https://github.com/michelbierlaire/optimization

Cancelled totally We select n trains from those that are cancelled at their first station.

The **Station**, **Section** and **Section Track** operators are designed to return significantly more trains, allowing for more substantial modifications to the solution.

4.2.2 Repair Operators

We currently restrict our algorithm to feasible solutions. All our *repair* operators therefore return a feasible timetable for the trains selected by the *destroy* operators.

Cancel completely This operator cancels the train at its first event.

Cancel partially This operator reverts the train to its schedule, and cancels it at the last station with a shunting yard before its first conflict *(a conflict being another train or a disruption on the track it was scheduled on).*

Delay This operator starts by reverting the train to its schedule, then resolves conflicts by delaying the train until the next free slot on the track.

Change track and/or delay This operator starts by reverting the train to its schedule, then resolves conflicts by changing the track and/or delaying the train, choosing the track that would allow the train to arrive the earliest.

Add stops This operator replaces one random *pass-through* activity of the train with the corresponding *waiting* activity, effectively adding a stop to the train. It then reuses the **Change track and/or delay** mechanism to ensure the solution is feasible.

Remove stops Similarly, this operator replaces one random *waiting* activity of the train with the corresponding *pass-through* activity, effectively removing a stop from the train. It then reuses the **Change track and/or delay** mechanism to ensure the solution is feasible.

In any case, we do not allow any event to occur before its scheduled time. In addition, if an event ends up being delayed by more than 30 minutes, we cancel the train at the last shunting yard before that station.

In the case where there are multiple trains returned by the *destroy* operator, we start by removing all selected trains from the solution, then apply the *repair* operator to each of them in the order they were returned by the *destroy* operator (random in most cases, or the most/least used train first for the operators based on the passengers' paths).

5 Case study

5.1 Experimental context

We tested our passenger assignment algorithms and our ALNS operators on the simplified Dutch network instance used by Stefan Binder et al. (2017c). The network is represented in Figure 2. The numbers on the tracks are respectively the travel time (in minutes) and the distance (in kilometres). We consider two tracks in each station and on each section. Passenger demand is generated from probability distributions, with 55 passenger groups of 100 passengers (Stefan Binder et al. 2017c). The exogenous priority of the passenger groups is based on their desired start time (the group wanting to start first has higher priority).

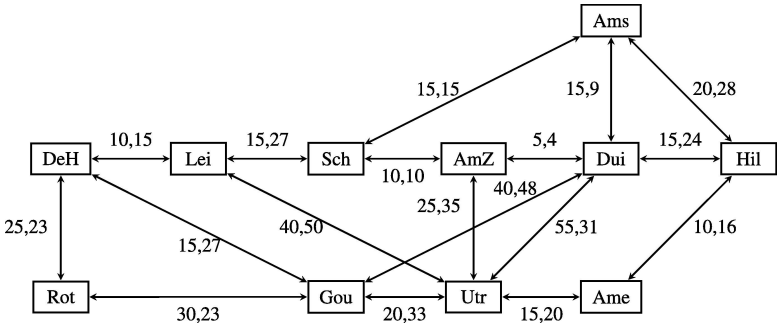


Figure 2: Dutch network-inspired instance (Stefan Binder et al. 2017c)

The original timetable is generated by an exact optimisation of the passenger costs and then the operational cost (in the ϵ-constrained method) with 15 trains. We then consider a disruption that completely blocks both tracks from Leiden (Lei) and Den Haag (DeH) for 40 minutes at the start of the studied time period.

Table 5 presents the general parameters of the problem on which we ran our tests.

Parameter	Symbol	Value	Unit
Time horizon	H	120	min
In-vehicle time weight	–	1	min/min
Waiting time weight	β_1	2.5	min/min
Transfer penalty	β_2	10	min/transfer
Early departure time weight	β_3	0.5	min/min
Late departure time weight	β_4	1	min/min
Cost per km	c_{km}	30	CHF/km
Penalty cost	P	$5 \cdot H = 600$	min
Minimum transfer time	–	2	min
Maximum transfer time	–	30	min
Minimum headway (passenger trains)	–	2	min
Minimum headway (freight trains)	–	3	min
Minimum separation time	–	2	min
Deviation cost weight (cancellation)	δ_1	50	–
Deviation cost weight (delay)	δ_3	5	–
Deviation cost weight (skipped stops)	δ_5	10	–
Deviation cost weight (added stops)	δ_6	1	–
Deviation cost weight (changed tracks)	δ_7	50	–
Train capacity	q	400	passengers
Start time	–	16:00 = 960	min

Table 5: General parameters used for our tests

Our results are compared with the exact solutions of the MILP implemented in Ricard et al. (2025). To generate a Pareto front, we run the task with multiple ϵ values.

Our operators only implement cancelling, delaying, changing tracks, or adding/removing a stop strategies. The only way to impact the operational cost with these strategies, as we have defined it, is to cancel a train, resulting in a direct improvement in the operational cost but losses in the other two objectives. In addition, since we currently accept a solution if it is Pareto dominant, and an improvement in one objective is Pareto

dominant, the "cancel" operators, which directly improve the operational cost, quickly receive a very high score. As such, we remove the operational cost from our objective functions, testing the ALNS heuristic only on the passenger cost and the deviation from the original schedule. However, the aim of the project is to include other recovery strategies, such as rerouting or emergency trains and buses, for which the operational cost would become more meaningful.

The experiment was conducted on the SCITAS computational infrastructure of EPFL, which runs `Intel(R) Xeon(R) Gold 6140 CPU @ 2.30GHz` processors. Each heuristic run was given 1 CPU and 5GB of RAM. We ran the heuristic algorithms with different values for the parameter N (100, 250, 500 and 1000), 10 times each.

5.2 Convergence

Here, we discuss the speed at which the algorithm produces solutions with good objective values. To do so, we plot in Figure 3 [page 15] the evolution of the best value of f_p and f_d produced by the algorithm at each iteration for all runs. We also defined a metric for the production of good solutions for both objectives: $\hat{f}_d + \hat{f}_p$, where $\hat{f}_i = \frac{f_i}{\min f_i}$ with f_i the minimum value of objective i in all the non-dominated solutions and $\min f_i$ the minimum final value of this quantity across all considered runs.

In terms of deviation from the original timetable (f_d), the values quickly converge to under 200 in most cases. The best value (255.0) is obtained only by 8 runs after a varying number of iterations (between 2 and 790). Seeing the difference in speeds at which the algorithm reduces the optimal f_d value, it shows a dependence on the initial choices of operators. After 600 iterations, however, the 10 runs with 1000 iterations were all within 20% of the best found value. In terms of passenger cost, a plateau arises at 326,000, under which many runs do not reach, even after 1000 iterations. The relative gap between the best found value and the maximum value after 1000 iterations is around 24%.

When looking at our combination of both objectives, we see a clear convergence to low values, meaning that our algorithm manages to find solutions optimal for both objectives in the same run. Quantitatively, the best possible value is 2.0, which would mean the algorithm found the best value for both objectives in the same run. After 1000 iterations the 10 runs are all within 21% of this value.

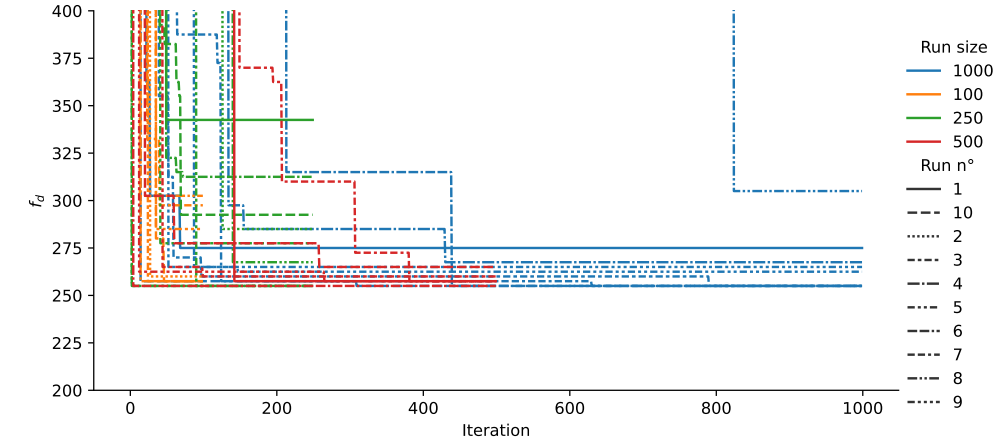
5.3 Non-dominated solutions

Here, we discuss the ability of the algorithm to produce a variety of non-dominated solutions, approximating the Pareto front. Table 6 shows the number of such solutions at the end of each run. As we can see, running for more iterations does increase the chance to find more solutions. This could mean that in the first iterations, our algorithm is not diversifying but rather finding new dominating solutions. Additionally, many runs produce only one non-dominated solution. This could be a consequence of the definition of our two objectives: indeed, the deviation from the original timetable and the passenger costs are similar in many terms.

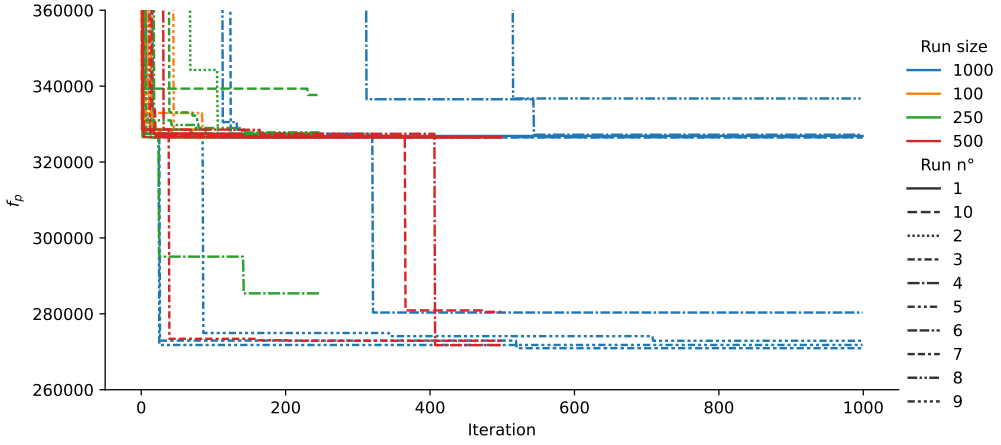
Run n° / Run size	1	2	3	4	5	6	7	8	9	10
100	1	2	1	1	3	1	2	2	2	2
250	1	2	1	3	2	3	2	1	1	6
500	1	1	3	3	1	1	2	1	2	3
1000	2	1	3	3	5	2	2	3	4	2

Table 6: Number of non-dominated solutions at the end of the run

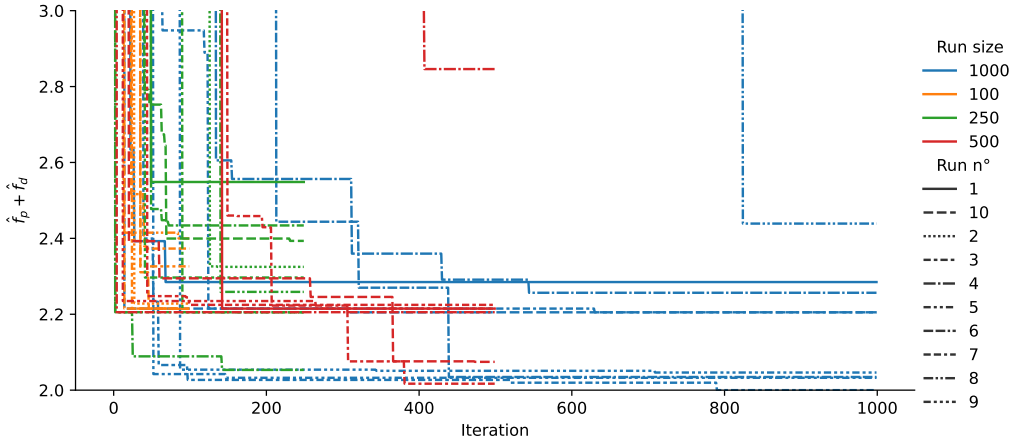
Figure 4 presents the non-dominated solutions in the (f_p, f_d) plane for each run. Run `1000-5`'s non-dominated set presents good solutions for each objective and three other non-dominated solutions which are more balanced (two of which share the same objective value). On the other hand, run `250-10` found many non-dominated solutions (6), but all of those would be dominated by the solutions found in run `250-2`.



(a) Deviation from the original timetable: f_d



(b) Passenger cost (at UE): f_p



(c) Both objectives: $\hat{f}_d + \hat{f}_p$

Figure 3: Convergence of the objective functions over the number of iterations

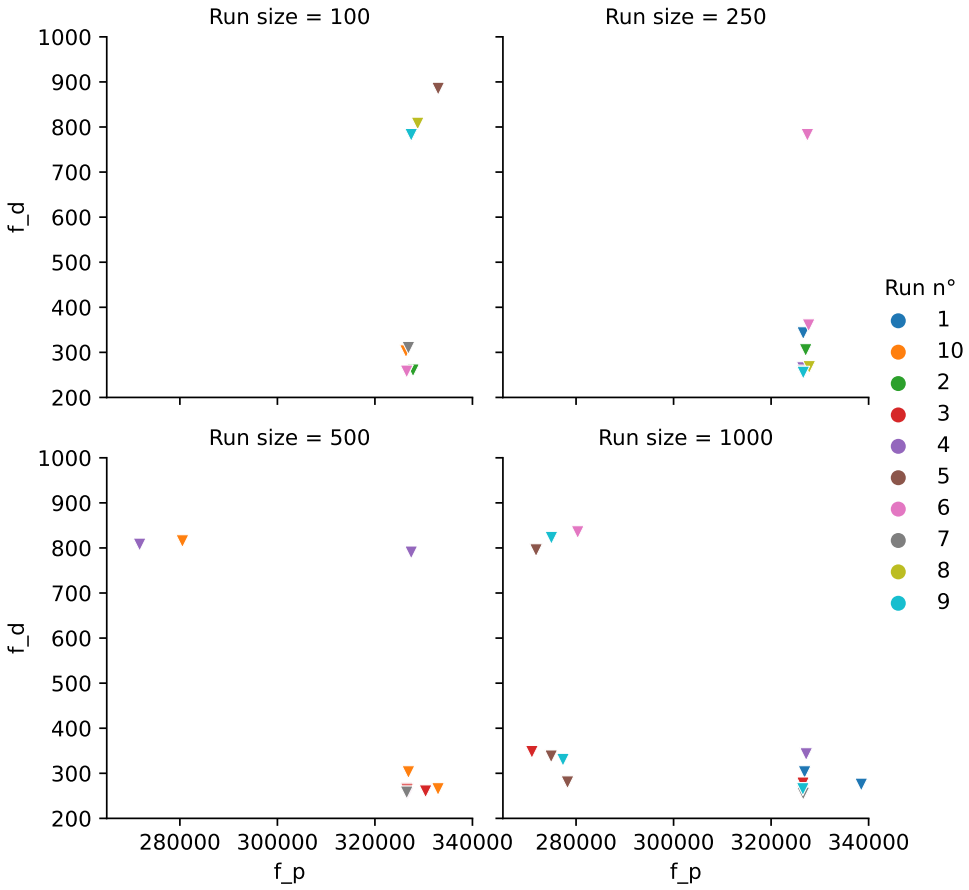


Figure 4: Non-dominated solutions of each run

The non-dominated points are compared with the exact Pareto front generated by the MILP in Figure 5, with run `1000-5` highlighted in green. It is important to keep in mind that the MILP computes the passenger cost at System Optimum (SO), while the heuristic computes a User Equilibrium (UE) passenger cost (or an approximation of it). As such, we also plot the passenger cost at UE of the exact solutions, for comparison purposes.

As we can see, the solutions from run `1000-5` generate a relatively good approximation of the Pareto front, even reaching values similar to the best SO values obtained by the MILP, which gives a lower bound on the best solution we could find. We observe that the plateau we were observing on Figure 3b corresponds to the passenger cost at UE of the MILP solutions, meaning our heuristic often reaches those solutions or equivalent solutions, but that there exist better solutions at User Equilibrium.

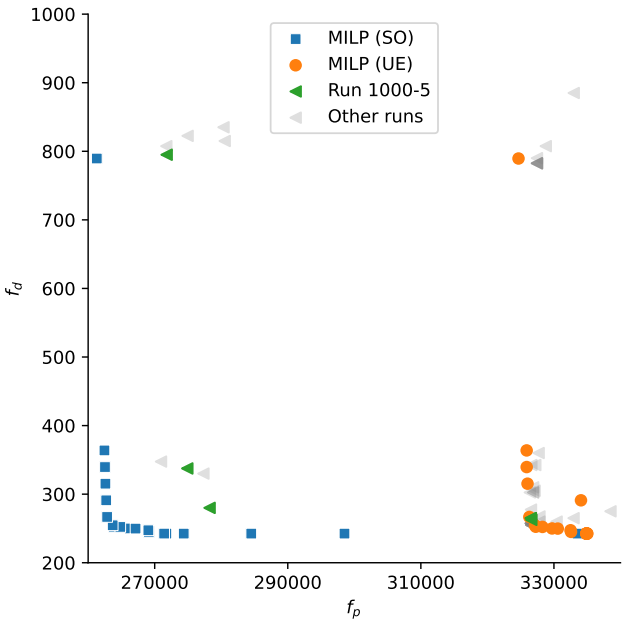


Figure 5: Comparison of the non-dominated solutions of the heuristic and the exact solutions of the MILP

5.4 Operators' performance

By taking the outer product of our destroy and our repair operators, we generated 77 different pairs of operators. In this subsection, we will analyse them, in order to identify the most efficient.

An interesting metric is the number of times these operators generated an accepted solution. Table 7 presents this number for each combination of repair and destroy operator, across all runs. Twenty-six operator pairs did not generate any accepted solution; and only twelve did so more than five times. The delay and delay track operators are the most efficient of the repair operators, which could be expected due to their capacity to modify the solution and their computational complexity. In terms of selection procedure (destroy operators), taking a train cancelled at least partially (`cancelled`) is the most efficient, which could have been expected : the result is likely to improve the objectives functions and has less chances of providing a degraded solution. The next most efficient is the `station` operator, which selects all trains passing through a station, rearranging the priorities between those trains in case of conflict.

Table 7 also presents the number of times each operator has been called. This number is similar between all operators, altough there is still a correlation with their performance. Two factors could explain this result : (1) We ran the algorithm 40 different times, with varying number of iterations. For the first iterations, all operators have the same probability of being taken, resulting in a "base" number of times each operator is called for each. (2) The scoring mechanism in the framework penalises any non-accepted result from the operator, and the reward for providing a good solution is the same as this penalty. Since in our case the acceptance rate is low (even for the best operators), the scores are not differentiating enough the most efficient operators.

repair operator / destroy operator	add one stop	cancel partially operator	cancel train completely	delay operator	delay track operator	remove one stop	**Total**
cancelled	- / 481	- / 495	2 / 493	16 / 519	22 / 530	- / 485	40 / 3003
cancelled totally	- / 478	1 / 494		11 / 509	35 / 564	- / 488	47 / 2533
max most used	3 / 510	- / 497	- / 497	1 / 507	- / 508	1 / 501	5 / 3020
mean less used	2 / 506	1 / 509	- / 506	3 / 506	4 / 510	- / 500	10 / 3037
mean most used	6 / 525	2 / 509	- / 493	4 / 512	3 / 506	- / 498	15 / 3043
min less used	- / 500	2 / 498	2 / 505	14 / 523	16 / 535	- / 510	34 / 3071
random selection	3 / 513	- / 502	2 / 506	5 / 507	- / 497	- / 510	10 / 3035
section	1 / 494	3 / 505	- / 487	3 / 492	11 / 508	2 / 498	20 / 2984
section track	3 / 504	1 / 506	- / 499	4 / 515	3 / 504	- / 500	11 / 3028
station	1 / 505	2 / 510	- / 496	13 / 526	18 / 544	2 / 502	36 / 3083
through disrupted section	4 / 507	- / 501	- / 502	1 / 500	5 / 504	4 / 501	14 / 3015
through disrupted tracks	2 / 505	1 / 501	- / 495	- / 501	4 / 510	2 / 505	9 / 3017
through disruption	1 / 505	- / 507	1 / 500	9 / 514	15 / 520	5 / 516	31 / 3062
Total	26 / 6533	13 / 6534	7 / 5979	84 / 6631	136 / 6740	16 / 6514	282 / 38931

Table 7: Number of accepted solutions per repair and destroy operator (over the number of times this pair has been called)

5.5 Computation time

In this subsection, we compare the heuristic and the exact approach in terms of time. At this stage, however, we implemented the operators to test their capability to generate good solutions but did not try to optimise the final results in terms of time.

Figure 6 [page 19] presents the evolution of the best value of f_p and f_d produced by the algorithm over time, for each run. The running times for the MILP are shown in Table 8. In the MILP, the parameter `MIP gap` represents an early stopping criterion, representing when the programme should stop, as a difference between the best objective value found and the best lower bound. Therefore, it represents the maximum possible difference

between the returned value and the exact solution.

MIP gap	ϵ	Objectives	Time [s]
0.25	0.02	$f_p \to f_d$	189
0.20	0.02	$f_p \to f_d$	218
0.15	0.02	$f_p \to f_d$	263
0.10	0.02	$f_p \to f_d$	527
0.09	0.02	$f_p \to f_d$	885
0.08	0.02	$f_p \to f_d$	1395
0.07	0.02	$f_p \to f_d$	946
0.06	0.02	$f_p \to f_d$	1444
0.05	0.02	$f_p \to f_d$	1297
0.04	0.02	$f_p \to f_d$	995
0.03	0.02	$f_p \to f_d$	1353
0.02	0.02	$f_p \to f_d$	1777
0.01	0.02	$f_p \to f_d$	2404
0	0.02	$f_p \to f_d$	2829

MIP gap	ϵ	Objectives	Time [s]
0.25	0.02	$f_d \to f_p$	18
0.20	0.02	$f_d \to f_p$	24
0.15	0.02	$f_d \to f_p$	18
0.10	0.02	$f_d \to f_p$	21
0.09	0.02	$f_d \to f_p$	22
0.08	0.02	$f_d \to f_p$	19
0.07	0.02	$f_d \to f_p$	20
0.06	0.02	$f_d \to f_p$	24
0.05	0.02	$f_d \to f_p$	27
0.04	0.02	$f_d \to f_p$	22
0.03	0.02	$f_d \to f_p$	29
0.02	0.02	$f_d \to f_p$	48
0.01	0.02	$f_d \to f_p$	51
0	0.02	$f_d \to f_p$	90

Table 8: Running times for the MILP

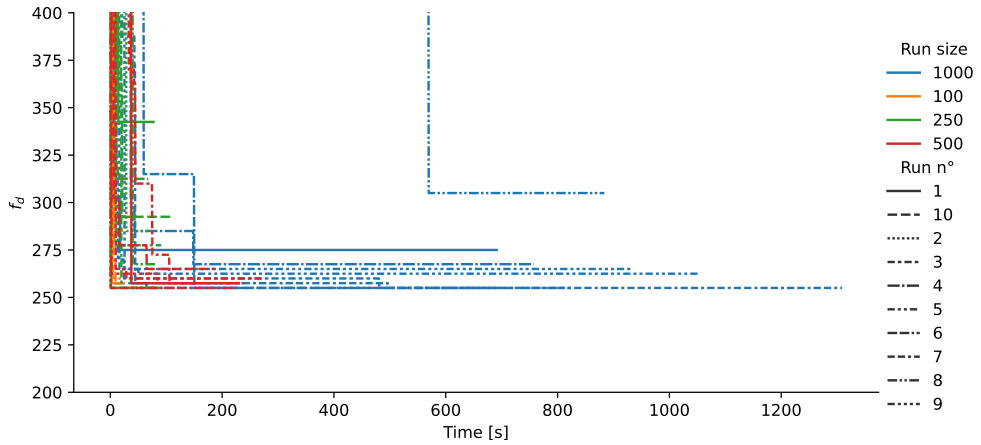
As we can see, the MILP produces better solutions faster than the heuristic, especially if we set f_d as the first objective. This yields several comments:

- The heuristic is fully implemented in Python, in contrast to the MILP solver which is implemented in low-level languages to reduce running times.
- The current implementation of the heuristic operators was developed with the goal of testing the capacity of these algorithms to generate good and diverse solutions. There is potential for optimisation in this regard, for example by using more the libraries `numpy` and `pandas`, which can greatly improve computational times. It is also possible to precompute some information and reuse it when the algorithm runs, which could further improve the running time.
- As we discussed, the scoring mechanism does not bring much adaptibility in its current form. Improvements to the adaptive layer of the framework would probably improve the quality of the results in the same computational time.
- The time per iteration increases greatly as the number of iterations increases. An hypothesis could be the mechanism that checks whether a given solution has already been explored that is taking more time as the number of explored solutions increase. As such, the framework could be modified to limit the overhead.

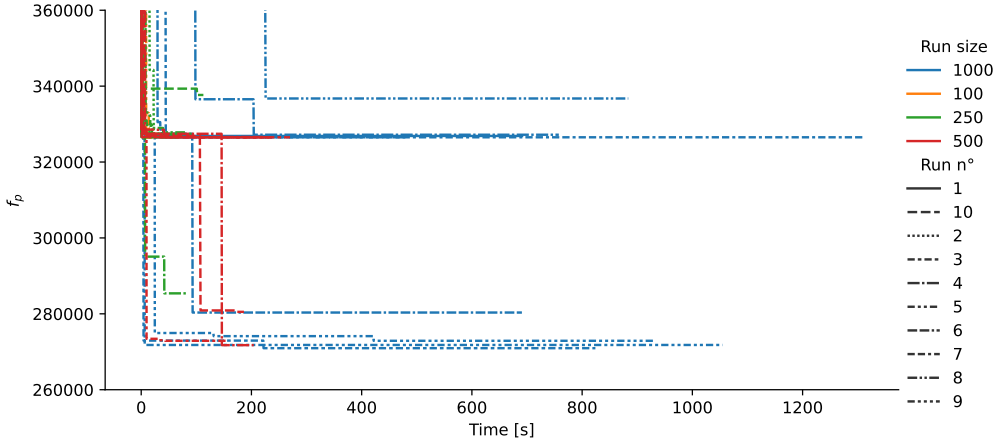
Iterations	min	mean	max
100	16	25	35
250	48	78	113
500	132	210	308
1000	479	847	1309

Table 9: Running times of the heuristic [s]

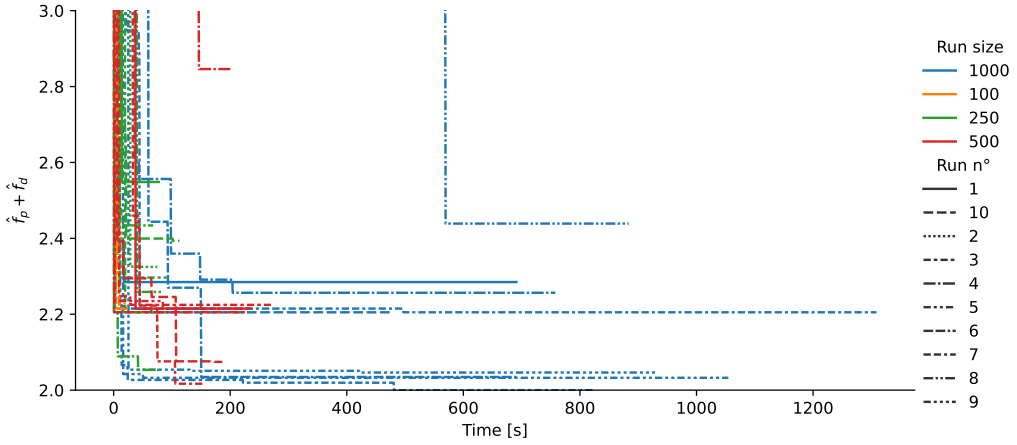
- The heuristic does not guarantee to produce good solutions in limited time, but some runs did produce good solutions already in the first 100 iterations. The 100 first iterations takes on average 25 seconds.
- The MILP was allocated 15 CPUs and is built to benefit from those through parallelisation, while the heuristic only used 1 CPU. One could run the heuristic 15 times with a lower number of iterations; it would probably produce diverse results in a short amount of time.



(a) Deviation from the original timetable: f_d



(b) Passenger cost (at UE): f_p



(c) Both objectives: $\hat{f}_d + \hat{f}_p$

Figure 6: Convergence of the objective functions over time

6 Conclusion

This work proposed a real-time ALNS heuristic for the multi-objective railway timetable rescheduling problem. The results obtained on a simplified Dutch network instance show promising performance in approaching good solutions for two objectives; however, we also identify opportunities for improvement in future work.

First, the algorithm should be subjected to a broader range of scenarios, involving different types of disruptions and networks, to ensure its adaptability and robustness.

Second, the current scoring mechanism, which drives the selection of operator pairs, and its parameters should be fine-tuned to the specifics of the algorithm. Furthermore, making the score sensitive to which objective is improved could help the algorithm avoid converging towards one-sided solutions.

Third, the set of destroy/repair operator pairings could be reduced. Excluding combinations lacking intuitive justification should be tested, starting with the least effective ones. This pruning could reduce noise and promote faster convergence.

The sensitivity of the algorithm to initial solutions also suggests that the current acceptance criterion may limit exploration. Introducing a mechanism that accepts non-improving solutions, or keep previously dominant solutions based on a probabilistic or temperature-controlled model (e.g., simulated annealing) could mitigate premature convergence and improve the algorithm's reliability.

Finally, the development of new repair operators should be considered, particularly those that respond directly to passenger demand patterns. For example, prioritising groups unable to find feasible routes or delaying trains to better accommodate peak demand could open up new areas of the solution space.

These extensions would contribute to a more realistic, adaptive, and efficient rescheduling tool for real-time railway operations.

References

Barrena, Eva, David Canca, Leandro C. Coelho, and Gilbert Laporte (2014). "Single-line rail rapid transit timetabling under dynamic passenger demand". In: *Transportation Research Part B: Methodological* 70, pp. 134–150. ISSN: 0191-2615. DOI: https://doi.org/10.1016/j.trb.2014.08.013. URL: https://www.sciencedirect.com/science/article/pii/S0191261514001544.

Binder, S., M.Y. Maknoon, Sh. Sharif Azadeh, and M. Bierlaire (2021). "Passenger-centric timetable rescheduling: A user equilibrium approach". In: *Transportation Research Part C: Emerging Technologies* 132, p. 103368. ISSN: 0968-090X. DOI: https://doi.org/10.1016/j.trc.2021.103368. URL: https://www.sciencedirect.com/science/article/pii/S0968090X21003703.

Binder, Stefan, Yousef Maknoon, and Michel Bierlaire (May 2017a). "Efficient exploration of the multiple objectives of the railway timetable rescheduling problem". In: *17th Swiss Transport Research Conference*.

— (2017b). "Exogenous priority rules for the capacitated passenger assignment problem". In: *Transportation Research Part B: Methodological* 105, pp. 19–42. ISSN: 0191-2615. DOI: https://doi.org/10.1016/j.trb.2017.08.022. URL: https://www.sciencedirect.com/science/article/pii/S0191261517301248.

— (2017c). "The multi-objective railway timetable rescheduling problem". In: *Transportation Research Part C: Emerging Technologies* 78, pp. 78–94. ISSN: 0968-090X. DOI: https://doi.org/10.1016/j.trc.2017.02.001. URL: https://www.sciencedirect.com/science/article/pii/S0968090X17300414.

Liao, Wenzhu, Liuyang Zhang, and Zhenzhen Wei (2020). "Multi-objective green meal delivery routing problem based on a two-stage solution strategy". In: *Journal of Cleaner Production* 258, p. 120627. ISSN: 0959-6526. DOI: https://doi.org/10.1016/j.jclepro.2020.120627. URL: https://www.sciencedirect.com/science/article/pii/S0959652620306740.

Mairy, Jean-Baptiste, Yves Deville, and Pascal Van Hentenryck (2011). "Reinforced adaptive large neighborhood search". In: *The Seventeenth International Conference on Principles and Practice of Constraint Programming (CP 2011)*. Springer Berlin/Heidelberg, Germany, p. 55.

Pisinger, David and Stefan Ropke (2019). "Large Neighborhood Search". In: *Handbook of Metaheuristics*. Ed. by Michel Gendreau and Jean-Yves Potvin. Cham: Springer International Publishing, pp. 99–127. ISBN: 978-3-319-91086-4. DOI: 10.1007/978-3-319-91086-4_4. URL: https://doi.org/10.1007/978-3-319-91086-4_4.

Ricard, Léa and Michel Bierlaire (Jan. 2025). *Modeling : Multi-objective disruption management for robust railway operations*. Tech. rep. [Not public, please contact the author if needed]. Ecole Polytechnique Fédérale de Lausanne (EPFL).

Rifai, Achmad P., Huu-Tho Nguyen, and Siti Zawiah Md Dawal (2016). "Multi-objective adaptive large neighborhood search for distributed reentrant permutation flow shop scheduling". In: *Applied Soft Computing* 40, pp. 42–57. ISSN: 1568-4946. DOI: https://doi.org/10.1016/j.asoc.2015.11.034. URL: https://www.sciencedirect.com/science/article/pii/S1568494615007528.

Ropke, Stefan and David Pisinger (2006). "An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows". In: *Transportation Science* 40.4, pp. 455–472. DOI: 10.1287/trsc.1050.0135. eprint: https://doi.org/10.1287/trsc.1050.0135. URL: https://doi.org/10.1287/trsc.1050.0135.

Santini, Alberto, Stefan Ropke, and Lars Magnus Hvattum (2018). "A comparison of acceptance criteria for the adaptive large neighbourhood search metaheuristic". In: *Journal of Heuristics* 24, pp. 783–815.

Shaw, Paul (1998). "Using Constraint Programming and Local Search Methods to Solve Vehicle Routing Problems". In: *Principles and Practice of Constraint Programming — CP98*. Ed. by Michael Maher and Jean-Francois Puget. Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 417–431. ISBN: 978-3-540-49481-2.

Thomas, Charles and Pierre Schaus (2018). "Revisiting the Self-adaptive Large Neighborhood Search". In: *Integration of Constraint Programming, Artificial Intelligence, and Operations Research*. Ed. by Willem-Jan van Hoeve. Cham: Springer International Publishing, pp. 557–566. ISBN: 978-3-319-93031-2.

Turkeš, Renata, Kenneth Sörensen, and Lars Magnus Hvattum (2021). "Meta-analysis of metaheuristics: Quantifying the effect of adaptiveness in adaptive large neighborhood search". In: *European Journal of Operational Research* 292.2, pp. 423–442. ISSN: 0377-2217. DOI: https://doi.org/10.1016/j.ejor.2020.10.045. URL: https://www.sciencedirect.com/science/article/pii/S037722172030936X.

Wikimedia Commons (2024). *File:Plan du réseau RER Vaud.svg*. [Online; accessed 10-mai-2025]. URL: %5Curl%7Bhttps://commons.wikimedia.org/w/index.php?title=File:Plan_du_r%C3%A9seau_RER_Vaud.svg&oldid=885062336%7D.

Wikipedia (2025). *Réseau express régional vaudois — Wikipedia, The Free Encyclopedia*. http://fr.wikipedia.org/w/index.php?title=R%C3%A9seau%20express%20r%C3%A9gional%20vaudois&oldid=225208057. [Online; accessed 10-May-2025].

Windras Mara, Setyo Tri, Rachmadi Norcahyo, Panca Jodiawan, Luluk Lusiantoro, and Achmad Pratama Rifai (2022). "A survey of adaptive large neighborhood search algorithms and applications". In: *Computers & Operations Research* 146, p. 105903. ISSN: 0305-0548. DOI: https://doi.org/10.1016/j.cor.2022.105903. URL: https://www.sciencedirect.com/science/article/pii/S0305054822001654.